Lab 7: Fuzzing for Input Validation Bugs

Sreekar Bandi | Repo 43: L6_L7_43_warranty_claims | Code under test: Lab 7/warranty_claims.py

What Hypothesis found in the original code

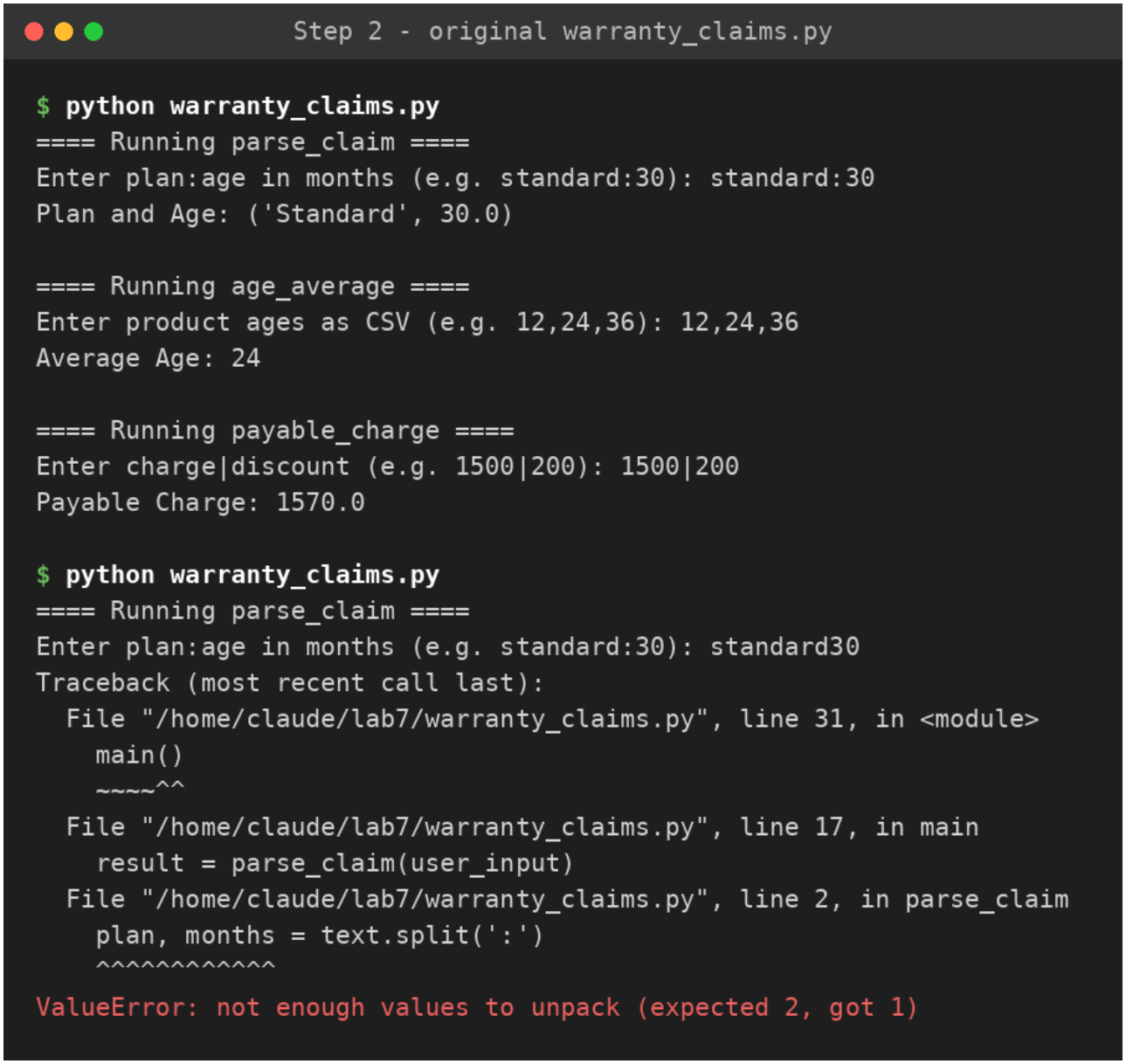
Function	Failing input	Exception
parse_claim	':'	ValueError: could not convert string to float: ''
parse_claim	'' (no colon)	ValueError: not enough values to unpack (expected 2, got 1)
parse_claim	None	AttributeError: 'NoneType' object has no attribute 'split'
age_average	'' (empty / blank element)	ValueError: invalid literal for int() with base 10: ''
age_average	None	AttributeError: 'NoneType' object has no attribute 'split'
payable_charge	'|'	ValueError: could not convert string to float: ''
payable_charge	'' (no separator)	ValueError: not enough values to unpack (expected 2, got 1)
payable_charge	None	AttributeError: 'NoneType' object has no attribute 'split'

How the code was hardened (valid input gives exactly the same output as before)

- Every function now checks the input is a string and has exactly the expected separators, instead of unpacking blindly.
- Conversions (float / int) are wrapped so bad text returns None instead of raising; NaN, infinity and overflow are rejected.
- Negative months, ages, charges and discounts are rejected, and an empty plan name is rejected.
- main() prints a short 'Invalid input' message when a function returns None, so the program no longer crashes.
- test_warranty_claims.py: three @given(st.text() | st.none()) no-crash tests plus three valid-input property tests.
- Result: 3 failed / 3 passed on the original code; 6 passed on the fixed code (also with 8 different seeds).

Step 2: run the original application

Command: python warranty_claims.py



Top: valid input works. Bottom: a malformed answer crashes the program with a traceback (ValueError).

Step 4: run test_warranty_claims.py against the original code (1/2)

Command: pytest test_warranty_claims.py

Step 4 - test_warranty_claims.py against the original code (1/2)

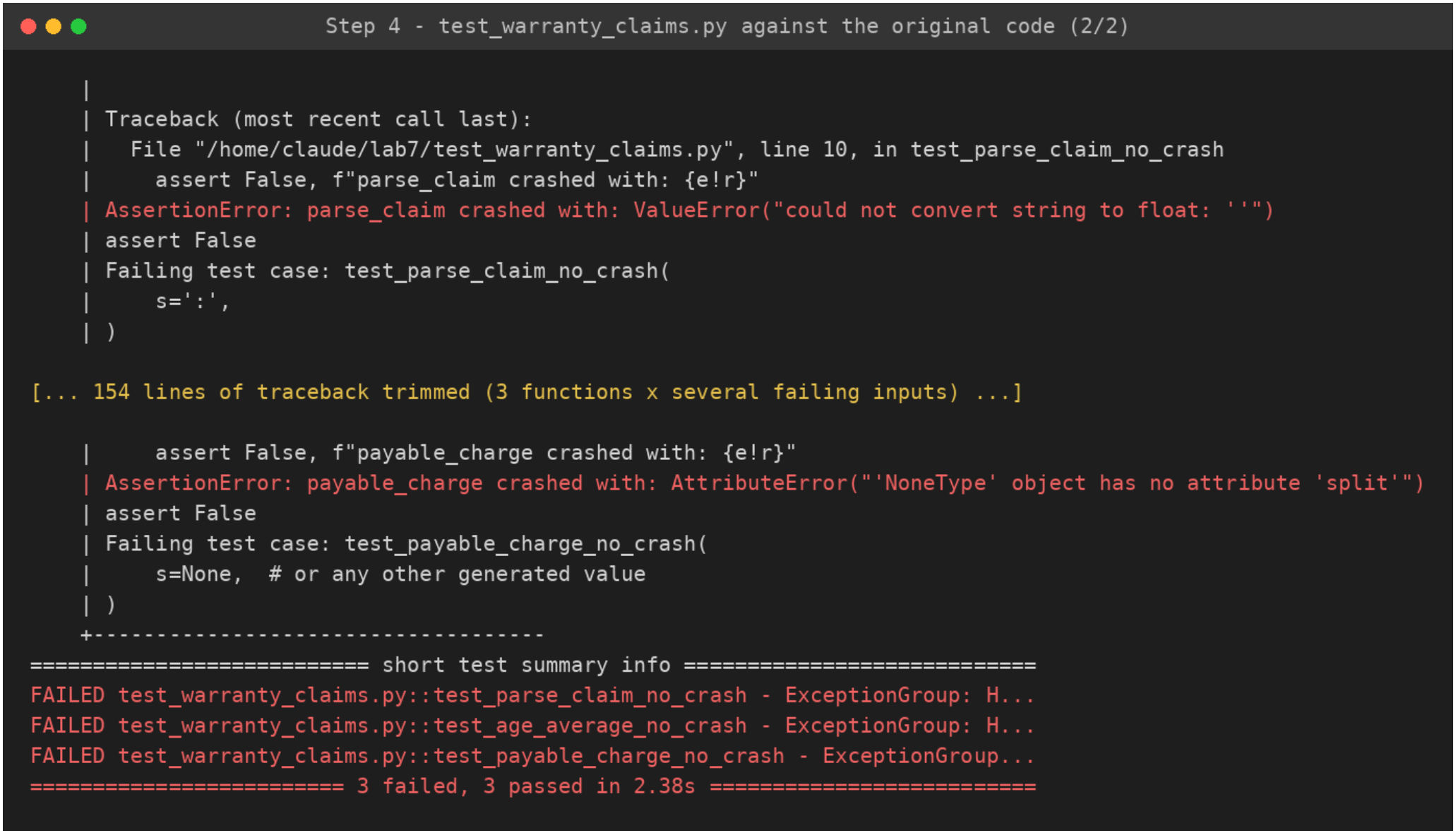
```
$ pytest test_warranty_claims.py
============================= test session starts ==============================
platform linux -- Python 3.13.18, pytest-9.2.0, pluggy-1.7.0
rootdir: /home/claude/lab7
plugins: hypothesis-6.170.15
collected 6 items

test_warranty_claims.py FFF...                                           [100%]

=================================== FAILURES ===================================
_____________________ test_parse_claim_no_crash ______________________
  + Exception Group Traceback (most recent call last):
  |   File "/home/claude/lab7/test_warranty_claims.py", line 6, in test_parse_claim_no_crash
  |     def test_parse_claim_no_crash(s):
  |                    ^^^
  | ExceptionGroup: Hypothesis found 3 distinct failures. (3 sub-exceptions)
  +-+---------------- 1 ----------------
    | Traceback (most recent call last):
    |   File "/home/claude/lab7/test_warranty_claims.py", line 8, in test_parse_claim_no_crash
    |     parse_claim(s)
    |     ~~~~~~~~~~~^^^
    |   File "/home/claude/lab7/warranty_claims.py", line 3, in parse_claim
    |     return plan.strip().title(), float(months)
    |                                  ~~~~~^^^^^^^^
    | ValueError: could not convert string to float: ''
    |
    | During handling of the above exception, another exception occurred:
```

Step 4: run test_warranty_claims.py against the original code (2/2)

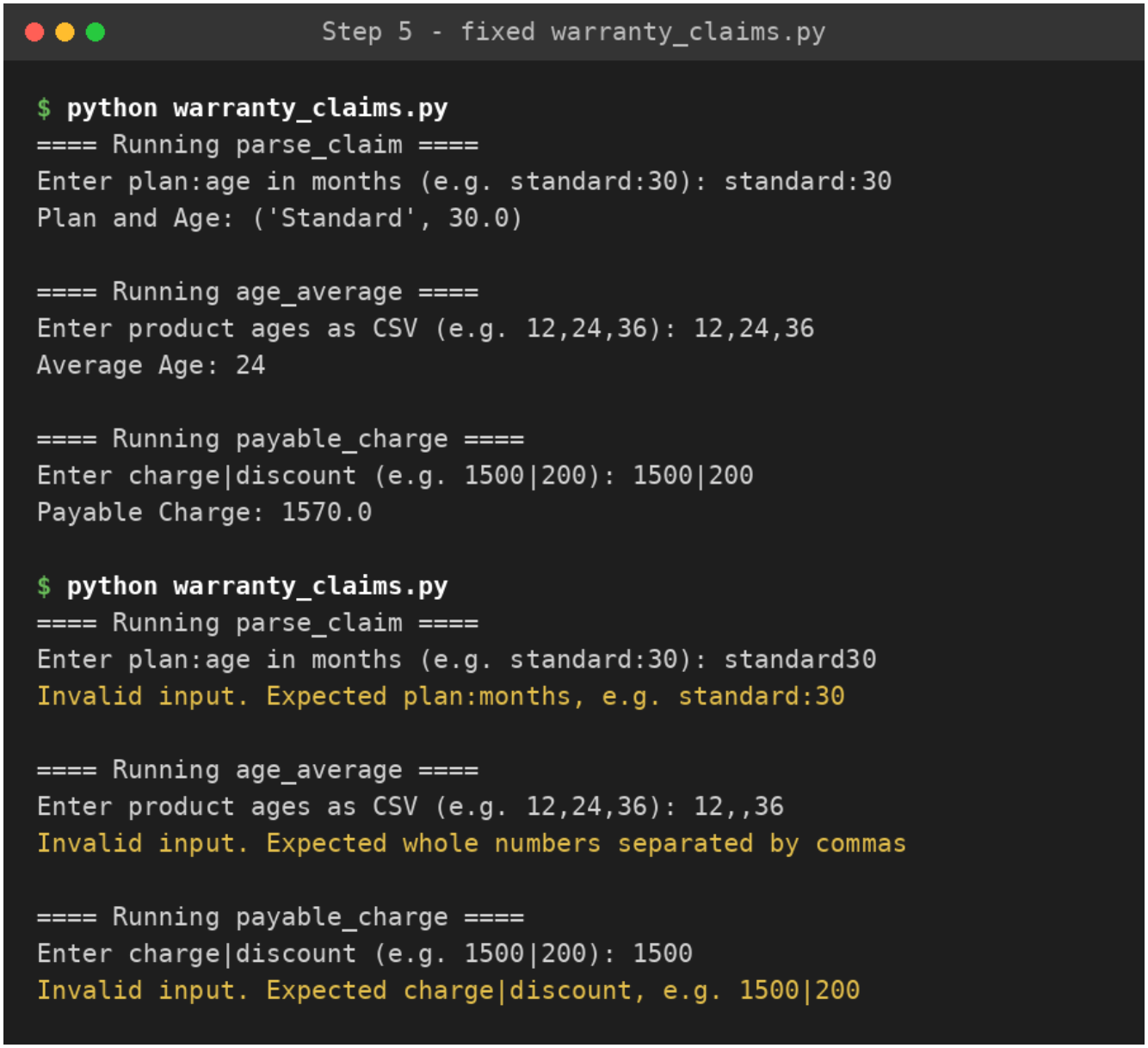
Command: pytest test_warranty_claims.py



3 failed, 3 passed: the three fuzz tests fail (Hypothesis found 8 crashing inputs); the valid-input tests pass.

Step 5: run the fixed application

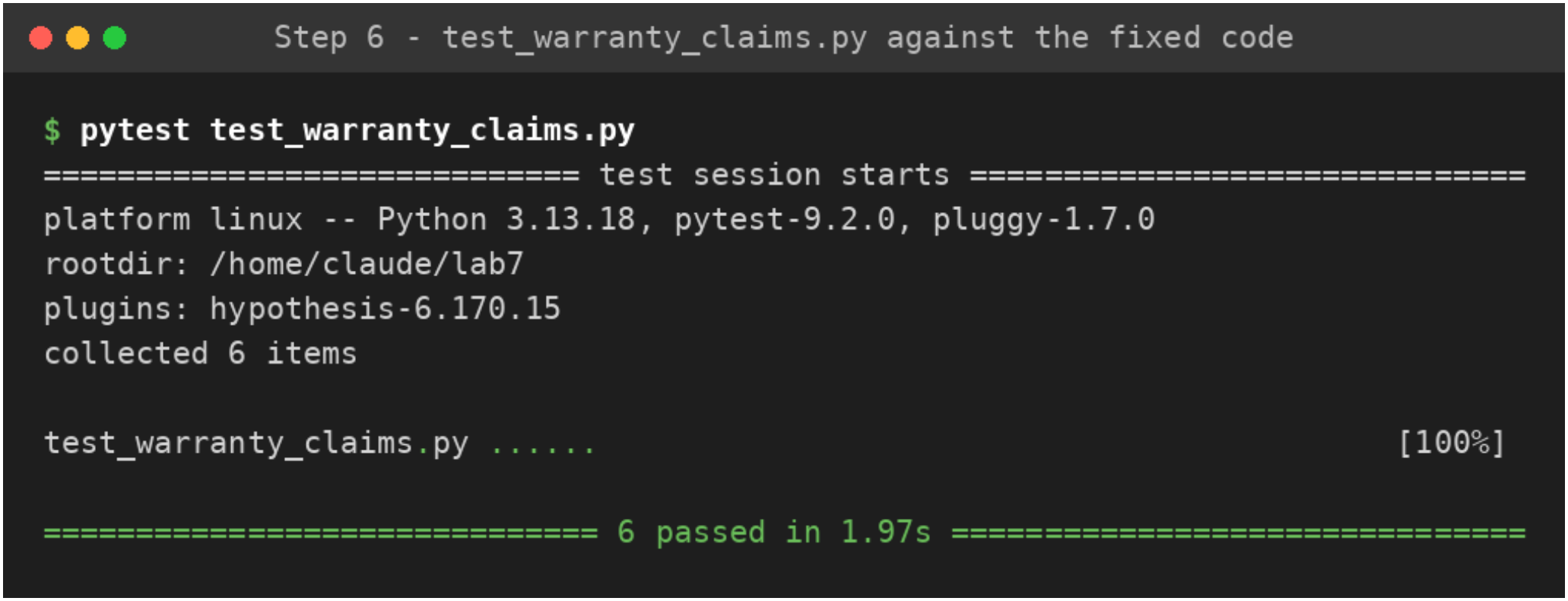
Command: python warranty_claims.py



Top: valid input gives the same output as before. Bottom: the same bad answers now print 'Invalid input'.

Step 6: re-run test_warranty_claims.py against the fixed code

Command: pytest test_warranty_claims.py



6 passed: no crash for any text or None input, and valid input is still processed correctly.